CSEN 146: Computer Networks
Lab 5: Stop and Wait for an Unreliable Channel with Error and Loss

Objective

1. To build a Stop and Wait reliable protocol on top of UDP to provide a reliable transport service while considering error and loss

TFv2 – Stop and Wait for an Unreliable Channel, with Loss

In Labs 3 and 4, you have developed a TCP and a UDP client/server to transfer files. Let us name it as TFv1. To ensure a reliable transfer over UDP, you need to implement a reliable Stop and Wait protocol. This version of file transfer is then TFv2

TFv2 will be built on top of UDP, and it is supposed to provide a reliable transport service to TFv1. Messages are sent one at a time, and each message needs to be acknowledged when received before a new message can be sent. TFv2, therefore, implements a reliable file transfer using the Stop and Wait protocol rdt3.0, which examines whether or not a packet and/or an ACK is in error or lost.

TFv2 consists of a client and a server. Communication is unidirectional, i.e., data flows from the client to the server. The server starts first and waits for messages. The client starts the communication. Messages have seq number 0 or 1. Before sending each message, a 1-byte checksum is calculated and added to the header. After sending each message, the client starts a timer and waits for a corresponding ACK. When it arrives, if it is not the corresponding ACK (or if the checksum does not match), the message is sent again. If it is the corresponding ACK, the client changes state and returns to the application, which can now send one more message. If a response is not received within the time set, the client will send the message again. This means TFv2 blocks on writes until an ACK is received or after a timeout.

After receiving a message, the server checks its checksum. If the message is correct and has the right seq number, the server sends an ACK0 or ACK1 message (according to the seq number) to the client, changes the state accordingly, and delivers data to the application.

The protocol should deal properly with duplicate data messages and duplicate ACK messages. Follow the FSM in the lecture notes and the textbook!

Protocol

HEADER

```
    seq_ack    int (32 bits)    // SEQ for data and ACK for Acknowledgement
    len        int (32 bits)    // Length of the data in bytes (zero for ACKS)
    checksum   int (32 bits)        // Checksum calculated (by byte)
```

PACKET

```
    header
    data       char (10 bytes)
```

SENDER

- Member seq_ack is used as SEQ, and the data is in member data.
- Each packet may have 10 or fewer bytes of data, and the sender only sends the necessary bytes.
- After transmitting the file, a packet with no data (len = 0) is sent to notify the receiver that the file is complete.

RECEIVER

- Member seq_ack is used as ACK, and data is empty (len = 0).

TFv2 message contains the header and the application data. No reordering is necessary since TFv2 sends the exact message given by the application one by one.

Checksum

The checksum must be calculated for messages from the server and client. The simplest checksum algorithm is the so-called longitudinal parity check, which breaks the data into "bytes" with a fixed number of 8 bits and then

computes the exclusive or (XOR) of all those bytes. The result is appended to the message as an extra byte. You may use the following code snippet to calculate the checksum.

```
int checksum = 0;
char *ptr = (char *)&packet;
char *end = ptr + sizeof(Header) + sizeof(packet)
while (ptr < end)
   checksum ^= *ptr++;
```

To verify your protocol, use the result of a random function to decide whether to send the right checksum or just zero. This will fake the error effect.

After sending each message, the client starts a timer using `select()`. If the return of `select()` is zero, there is no data, and the client needs to retransmit, restart the timer, and call select again. If select returns non-zero, there is data, so the client calls `recvfrom()` to receive the ACK and then processes it. If the ACK is not corrupted and the ACK number is correct, the client can send the next message.

After receiving each message, the server checks its checksum. If the message is correct and the seq number is right, the server sends an ACK message with the right seq number, and the data is ready to be written to the file. Otherwise, the server repeats the last ACK message and waits to receive a message again. So, your server code will be the same as the server code in Lab5. You only need to simulate a drop of ACK.

To verify your protocol, use the result of a random function to decide to send or skip a message, to decide to send or skip an ACK message (only change to the server), and to decide whether to send the right checksum or just zero. This will fake the error and loss effect.

SELECT

This is an example of how to use `select( )`:

```
// local variables needed
struct  timeval tv; // timer
int rv;     // select returned value

// set up reads file descriptor at the beginning of the function to be checked for being ready to read
fd_set  readfds;
fcntl (sock, F_SETFL, O_NONBLOCK);
…
// start before calling select
FD_ZERO (&readfds); //initializes readfds to have zero bits
FD_SET (sock, &readfds); //sets readfds bit

// set the timer
tv.tv_sec = 1;
tv.tv_usec = 0;

// call select (), which returns the number of ready descriptors contained in the bit masks.
// if the time limit expires, select returns zero and set errno
rv = select (sock + 1, &readfds, NULL, NULL, &tv);

if (rv == 0)
{
   // timeout, no data
}
else if (rv == 1)
{
   //There is data to be received
}
```

Important note

The server closes the file and terminates execution after the message with zero bytes arrives. If the ACK sent for that last message does not make it to the client, the client will keep resending it forever to a non-responding server. To avoid that, the client will start a counter after sending a message with zero bytes and will only resend that last message 3 times. After 3 times, it will return to the main function.

Requirements to complete the lab

Show the TA the correct execution of the programs you wrote and upload the source code to Camino.

Be sure to retain copies (machine and/or printed) of your source code. You will want these for study purposes and to resolve any grading questions (should they arise)

Please start each program with a descriptive block that includes minimally the following information:

```
/*
* Name: <your name>
* Date:
* Title: Lab5 - ….
* Description: This program … <you should
* complete an appropriate description here.>
*/
```